

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Modeling and simulation of flexible systems: A tutorial

Cristóbal Ponce

Assistant Professor
Department of Mechanical Engineering
Universidad Técnica Federico Santa María, Chile.

PHS models

- **PHS model (1):**

$$\underbrace{\begin{bmatrix}\dot{p}\\ \dot{r}\end{bmatrix}}_{\dot{z}} = \underbrace{\begin{bmatrix}0 & -I\\ I & 0\end{bmatrix}}_{J=-J^\top}\underbrace{\begin{bmatrix}\mathcal{M}^{-1}p\\ \mathcal{F}^*(\mathcal{K}_\epsilon \mathcal{F} r)\end{bmatrix}}_{\delta_z H(z)} \quad , \ \forall \mathtt{X} \in \Omega$$

$$H(z) = \int_\Omega \left[\frac{1}{2}p^\top \mathcal{M}^{-1} p + \frac{1}{2}(\mathcal{F}r)^\top \mathcal{K}_\epsilon (\mathcal{F}r)\right] d\mathtt{X}$$

- **PHS model (2):**

$$\underbrace{\begin{bmatrix}\dot{p}\\ \dot{\epsilon}\\ \dot{r}\end{bmatrix}}_{\dot{x}} = \underbrace{\begin{bmatrix}0 & -\mathcal{F}^* & -I\\ \mathcal{F} & 0 & 0\\ I & 0 & 0\end{bmatrix}}_{\mathcal{J}=-\mathcal{J}^*}\underbrace{\begin{bmatrix}\mathcal{M}^{-1}p\\ \mathcal{K}_\epsilon \epsilon\\ 0\end{bmatrix}}_{\delta_x H(x)} \quad , \ \forall \mathtt{X} \in \Omega$$

$$H(x) = \int_\Omega \left[\frac{1}{2}p^\top \mathcal{M}^{-1} p + \frac{1}{2}\epsilon^\top \mathcal{K}_\epsilon \epsilon\right] d\mathtt{X}$$

Note: The models are defined by Ω, $\mathcal{M}$, $\mathcal{K}_\epsilon$, $\mathcal{F}$, and the BCs.

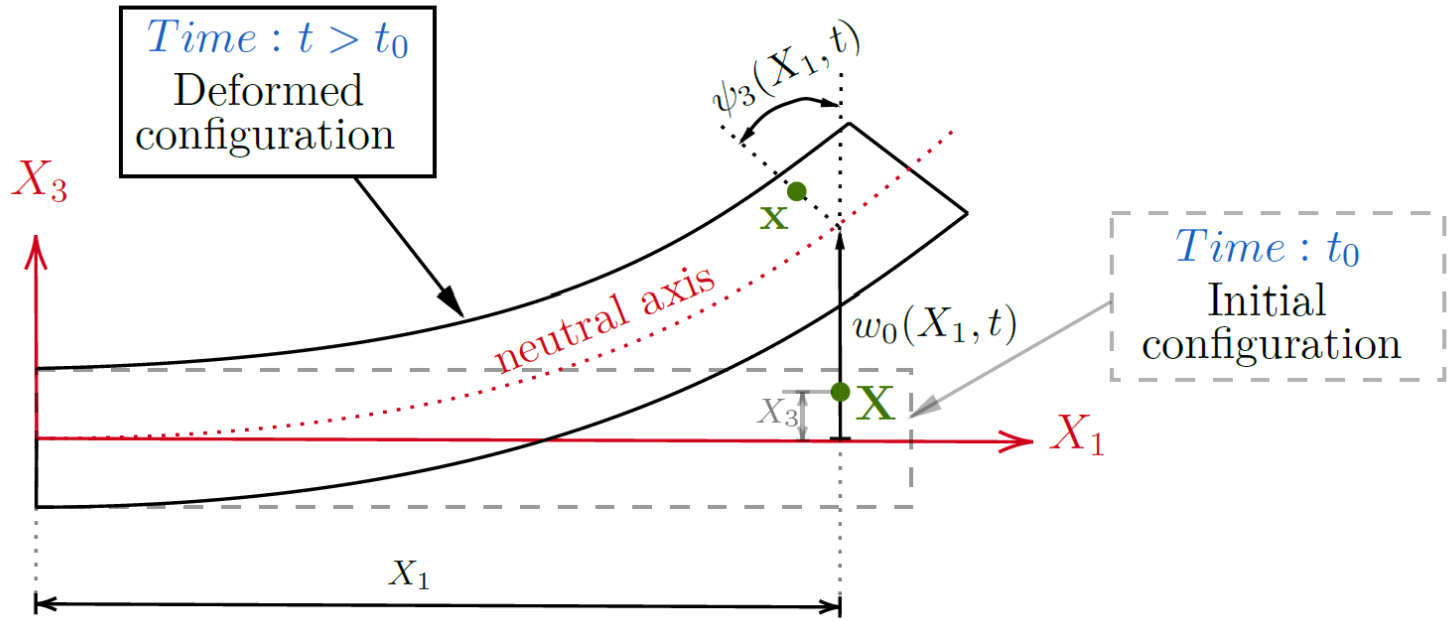
Outline

1. MATLAB code - FEM 1D

2. MATLAB code - FEM 2D

1. MATLAB code - FEM 1D

Example: Planar Timoshenko beam



$$r = \begin{bmatrix} \psi_3 \\ w_0 \end{bmatrix}, \quad \mathcal{F} = \begin{bmatrix} \partial_1 & 0 \\ -1 & \partial_1 \end{bmatrix}, \quad \mathcal{M} = \begin{bmatrix} \rho I_0 & 0 \\ 0 & \rho A_0 \end{bmatrix}, \quad \mathcal{K}_\epsilon = \begin{bmatrix} EI_0 & 0 \\ 0 & \kappa G A_0 \end{bmatrix}$$

where the differential operator $\mathcal{F}$ is decomposed as:

$$\mathcal{F} = \underbrace{\begin{bmatrix} 0 & 0 \\ -1 & 0 \end{bmatrix}}_{F_0} + \underbrace{\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}}_{F_1} \partial_1$$

MATLAB script

```
% Physical properties
rho   = 7800;          % Density of the material
E     = 2.1e11;        % Young's modulus
v     = 0.3;           % Poisson ratio
G     = E/(2*(1+v));   % Shear modulus
kappa = 5/6;           % Correction factor
L     = 80/100;        % Length of the beam
b     = 3/100;         % Width of the beam
h     = 1/1000;        % Thickness of the beam
A0    = b*h;           % Cross section area
I0    = b*(h^3)/12;    % Moment of inertia of the cross section

% Infinite-dimensional model (Planar Timoshenko beam)
prop.dom = [0 L];
prop.M  = @(x) [rho*I0 0;0 rho*A0];
prop.Ke = @(x) [E*I0 0;0 kappa*G*A0];
prop.F0 = [0 0;-1 0];
prop.F1 = [1 0;0 1];

% Boundary conditions
prop.Dirichlet = prop.dom(1);  % Coordinate where Dirichlet BC are applied
prop.Neumann   = prop.dom(2);  % Coordinate where Neumann BC are applied
prop.Didx = [0 0];             % Logical index relative to which generalized
                            ... velocities are applied as control inputs.
prop.Nidx = [0 1];             % Logical index relative to which generalized
                            ... tractions are applied as control inputs.
```

```
%% Finite-dimensional PHS model

Nelements = 30;        % Number of elements
method    = 'mFEM';    % sFEM: Hamilton's principle  |   mFEM: Hu-Washizu principle
TAG_mFEM  = 'C-D';     % 'C-C': strains continuous   |  'C-D': strains discontinuous

model = femPHS1D(prop,Nelements,TAG_mFEM);

J  = model.(method).J;
Q  = model.(method).Q;
G  = model.(method).G;
```

Command Window

```
model =
   struct with fields:

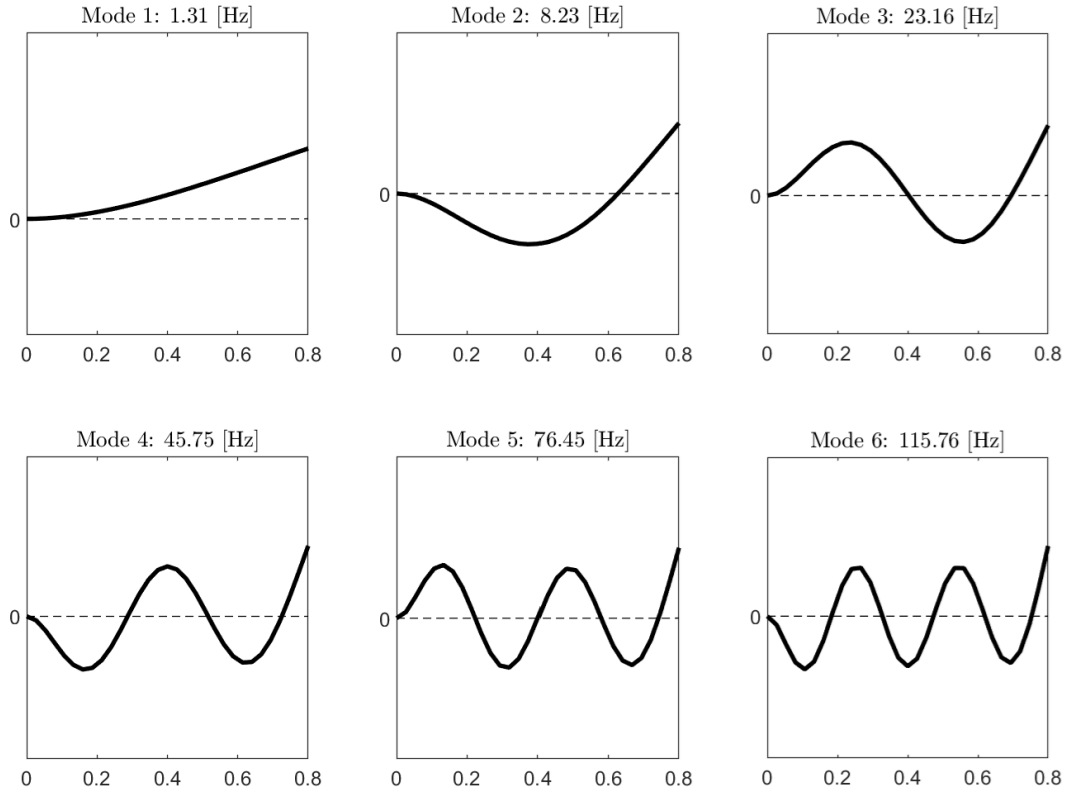
        sFEM: [1×1 struct]
        mFEM: [1×1 struct]
    TAG_mFEM: 'C-D'
        coor: [31×1 double]
          NE: 30
          NN: 31
      N_free: 60
     N_Omega: 62
     M_Omega: 60
    idx_sort: [62×1 double]
```

```
%% Eigenvalues and "eigenmodes"

lam = eig(full(J*Q))/2/pi; % Eigenvalues  [Hz]

imag_lam = round(imag(lam),2);
wn_fem   = sort(imag_lam(imag_lam>0))';

N_modes  = 6;
modesPlanarTimBeam(model,method,N_modes);
```



```
%% Time simulation

tspan = [0 2];  dt = 2e-4;
time  = tspan(1):dt:tspan(2);

% Input
Amp  = 1;  freq = 2*pi*8;
u    = @(t) Amp*sin(freq*t).*(t<0.5);

% Initial conditions
x0 = zeros(size(J,1),1);
x  = zeros(length(x0),length(time));
H  = zeros(length(time),1);

% Implicit midpoint rule
Id = speye(size(J,1));
A = (dt/2)*J*Q;
LHS_dec = decomposition(Id-A);
RHS_mat = Id + A;
for k = 1:length(time)
    H(k) = 0.5*x(:,k)'*Q*x(:,k);
    if k < length(time)
        u_mid = u(time(k) + dt/2);
        rhs = RHS_mat*x(:,k) + dt*(G*u_mid);
        x(:,k+1) = LHS_dec\rhs;
    end
end
```

```
%% Animation

coor     = model.coor;
NN       = model.NN;
N_free   = model.N_free;
M_Omega  = model.M_Omega;
idx_sort = model.idx_sort;

if isequal(method,'mFEM'); factor = 1; else; factor = 0; end

r_unsorted = x(N_free + factor*M_Omega + 1:end,:);
r = r_unsorted(idx_sort,:);
w = r(NN+1:end,:);

figure('color','w')
for i=1:10:length(time)

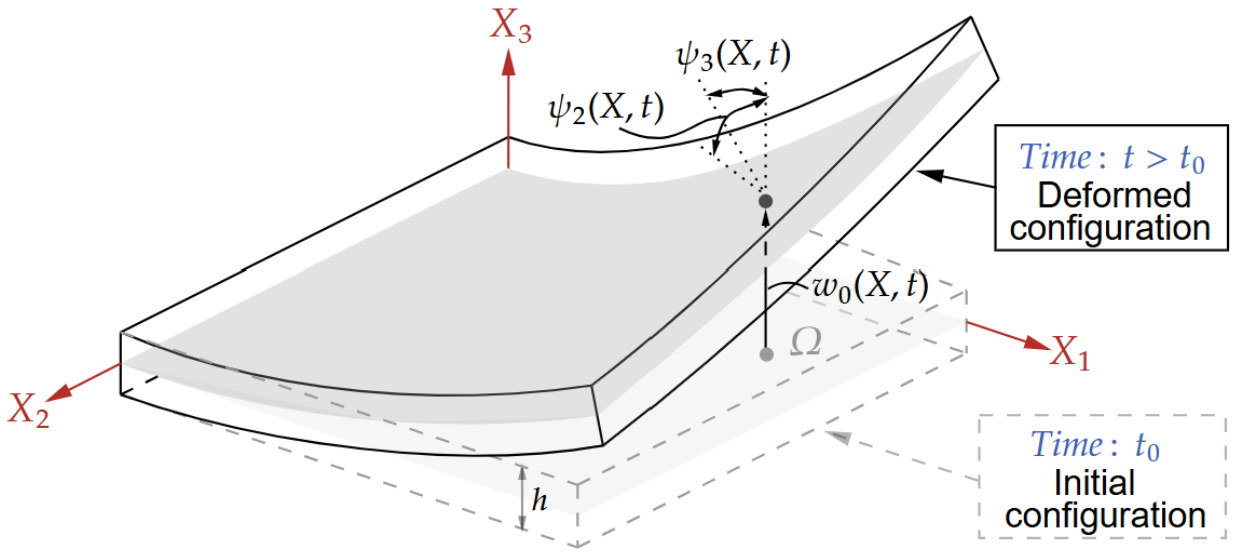
    plot(coor,w(:,i),'k','linewidth',2)
    xlabel('$x$','interpreter','latex','Fontsize',15)
    ylabel('$w_0(x,t)$','interpreter','latex','Fontsize',15)

    xlim([prop.dom(1) prop.dom(2)])
    ylim([-1 1]*(L/2))

    pause(0)
    clf
end
```

2. MATLAB code - FEM 2D

Example: Mindlin plate



$$r = \begin{bmatrix} \psi_2 \\ \psi_3 \\ w_0 \end{bmatrix}, \qquad \mathcal{M} = \begin{bmatrix} \rho\frac{h^3}{12} & 0 & 0 \\ 0 & \rho\frac{h^3}{12} & 0 \\ 0 & 0 & \rho h \end{bmatrix},$$

$$\mathcal{F} = \begin{bmatrix} \partial_1 & 0 & 0 \\ 0 & \partial_2 & 0 \\ \partial_2 & \partial_1 & 0 \\ -1 & 0 & \partial_1 \\ 0 & -1 & \partial_2 \end{bmatrix}, \quad \mathcal{K}_\epsilon = \begin{bmatrix} D & \nu D & 0 & 0 & 0 \\ \nu D & D & 0 & 0 & 0 \\ 0 & 0 & D\frac{(1-\nu)}{2} & 0 & 0 \\ 0 & 0 & 0 & \kappa G h & 0 \\ 0 & 0 & 0 & 0 & \kappa G h \end{bmatrix} \text{ with } D = \frac{E h^3}{12(1-\nu^2)}.$$

Create 2D Geometry: PDE Modeler

- saveDomain.m

```
% To create a new "Domain.mat" file, follow these steps:
% 1: Run the first section (pdetool) and create the 2D domain in the app.
% 2: Inside the PDE Modeler app, draw the domain and display the boundary.
%    Then, export the geometry (Boundary -> Export Decomposed Geometry, ...).
% 3: Exit the pdetool app (it is not necessary to save the file).
% 4: Choose a name for your "Domain.mat" file and run the second section.

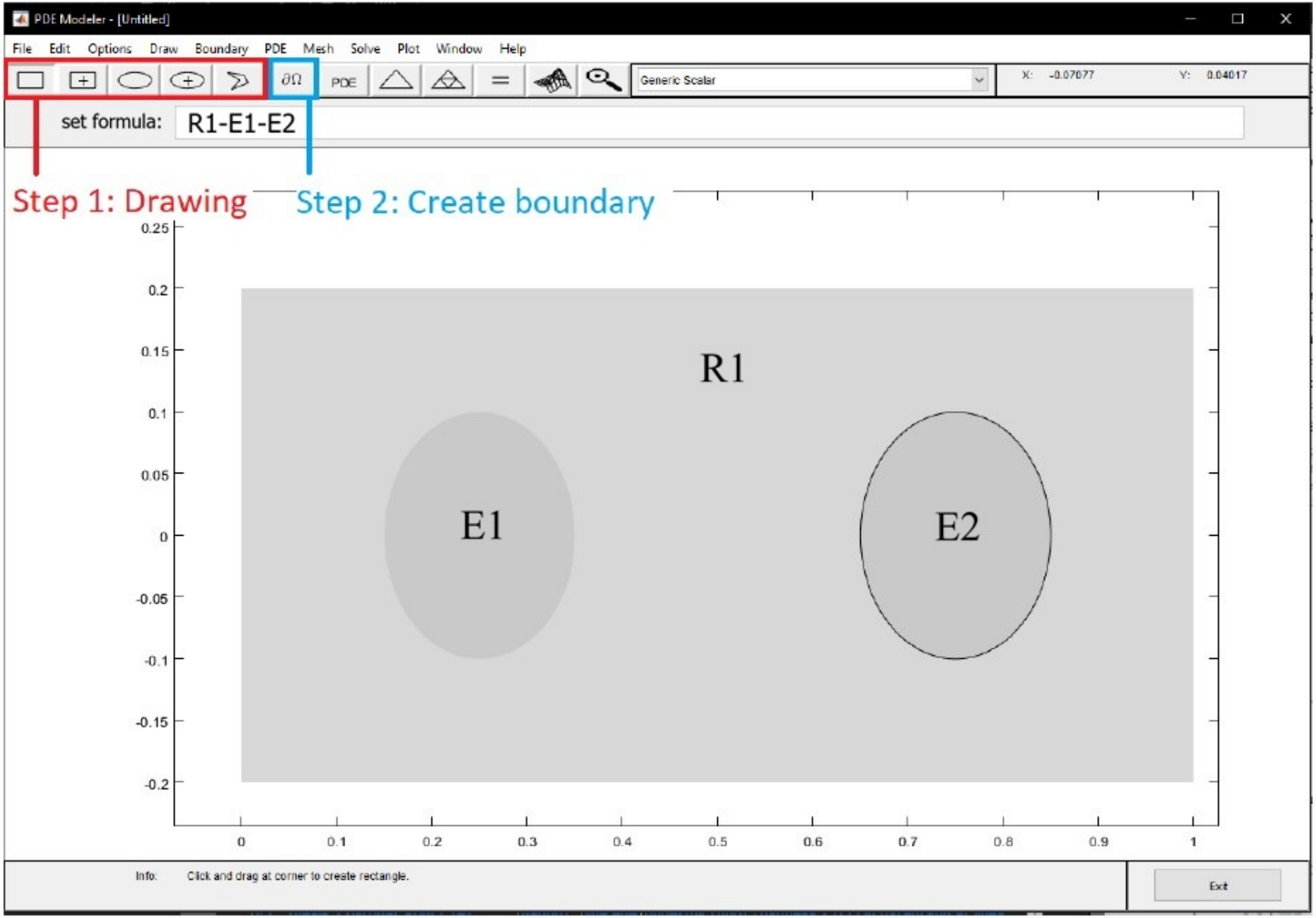
%% First section
clc , close all, clear all

pdetool;

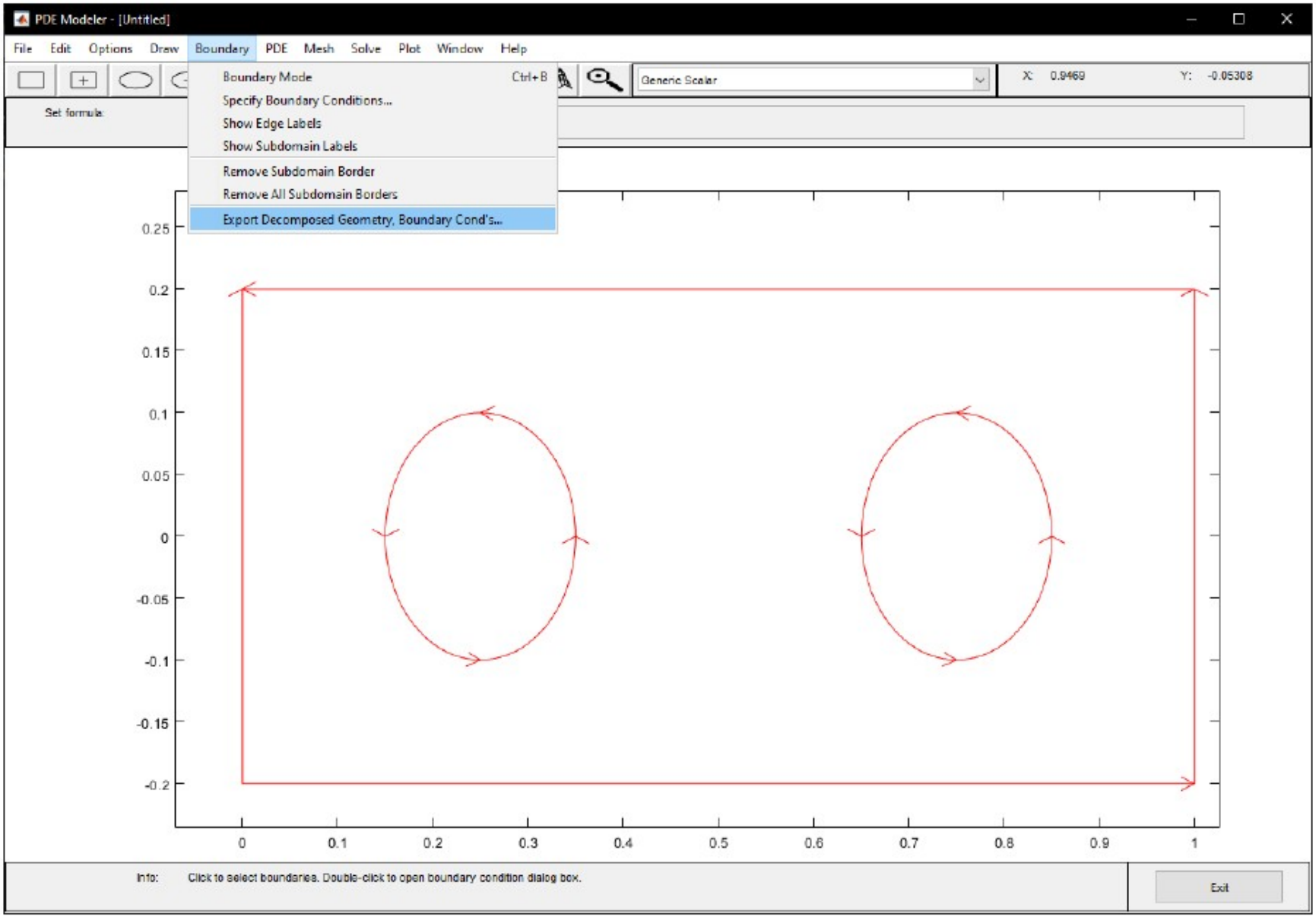
%% Second section

Name_domain = 'Domain3.mat';
save(Name_domain,'b','g')
```

Create 2D Geometry: PDE Modeler



Create 2D Geometry: PDE Modeler



Create 2D Geometry: PDE Modeler

- saveDomain.m

```
% To create a new "Domain.mat" file, follow these steps:
% 1: Run the first section (pdetool) and create the 2D domain in the app.
% 2: Inside the PDE Modeler app, draw the domain and display the boundary.
%    Then, export the geometry (Boundary -> Export Decomposed Geometry, ...).
% 3: Exit the pdetool app (it is not necessary to save the file).
% 4: Choose a name for your "Domain.mat" file and run the second section.

%% First section
clc , close all, clear all

pdetool;

%% Second section

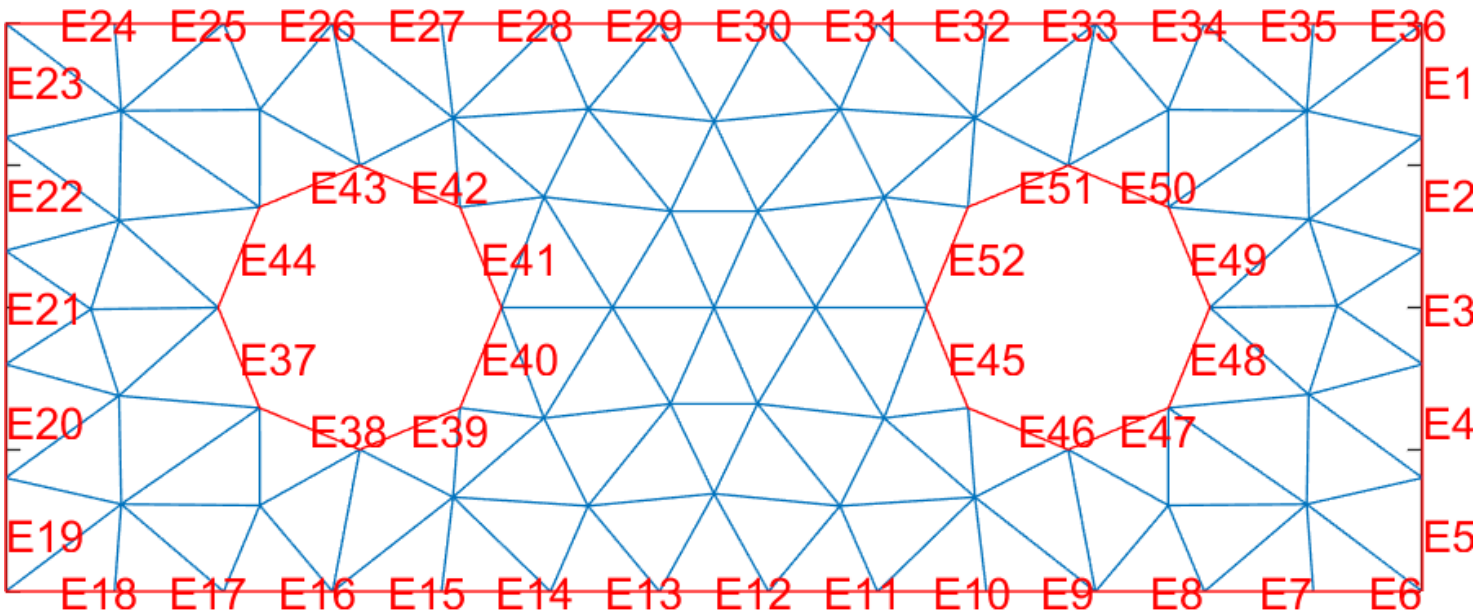
Name_domain = 'Domain3.mat';
save(Name_domain,'b','g')
```

MATLAB script

- gMesh2D.m / gMesh2D_aux.m

```
%% Generate mesh
DomainObj = load('Domain3.mat');
Hmax = 0.08;  % "Maximum" edge length: Hmax > 0. Automatic -> Hmax  = [];
tag_Nodes = 'off'; tag_Edges = 'on'; tag_Elements = 'off';

meshFEM = gMesh2D(DomainObj.g,Hmax,tag_Nodes,tag_Edges,tag_Elements);
% close(1);         % Comment to view the Figure with the mesh numbering
pause(0.5)
```

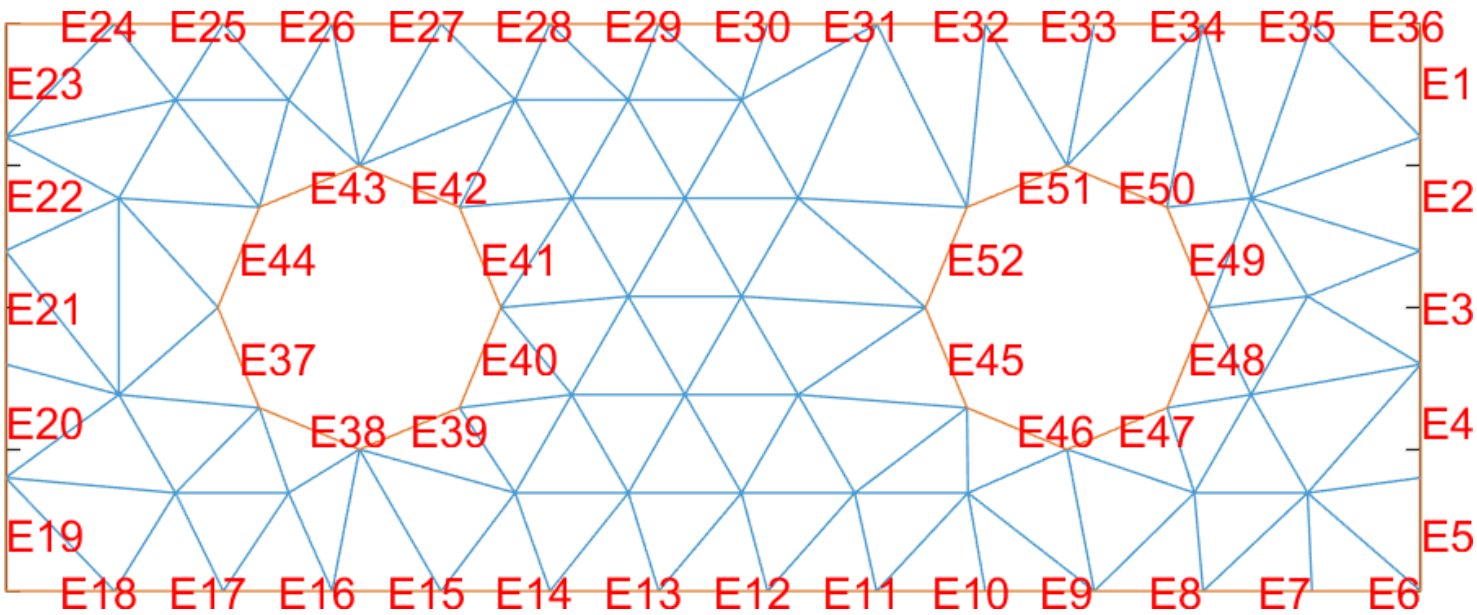


MATLAB script

- gMesh2D.m / gMesh2D_aux.m

```
%% Generate mesh
DomainObj = load('Domain3.mat');
Hmax = 0.08;  % "Maximum" edge length: Hmax > 0.
tag_Nodes = 'off'; tag_Edges = 'on'; tag_Elements = 'off';

meshFEM = gMesh2D_aux(DomainObj.g,Hmax,tag_Nodes,tag_Edges,tag_Elements);
% close(1);         % Comment to view the Figure with the mesh numbering
pause(0.5)
```



```
%% Physical properties
h   = 1/1000;       % thickness
rho = 7730;         % density of the material
I2  = (h^3)/12;     % second moment of inertia of the thickness
E   = 2.05e11;      % Young's modulus
v   = 0.3;          % Poisson's ratio
G   = E/(2*(1+v));  % Shear modulus

Cb = (E/(1-(v)^2))*[1 v 0; v 1 0; 0 0 (1-v)/2];  % Constitutive matrix for plane stress
Cs = 5/6*[G 0;0 G];                              % Constitutive matrix for shearing stress

% Infinite-dimensional model (Mindlin Plate)
prop.M  = @(x1,x2) rho*diag([I2,I2,h]);
prop.Ke = @(x1,x2) blkdiag(I2*Cb,h*Cs);
prop.F0 = [zeros(3,2) zeros(3,1);-eye(2) zeros(2,1)];
prop.F1 = [1 0 0;0 0 0;0 1 0;0 0 1;0 0 0];
prop.F2 = [0 0 0;0 1 0;1 0 0;0 0 0;0 0 1];

% Boundary conditions
prop.Dirichlet_Edges = (19:23)';   % Numbers of Global edges where Dirichlet BC are applied,
prop.Din  = [];                    % Edges where Dirichlet BC are applied as control inputs.
prop.Didx = [0 0 1];               % Logical index relative to which velocities are control inputs.
prop.Nin  = [1 2]';                % Edges where Neumann BC are applied as control inputs.
prop.Nidx = [0 0 1];               % Logical index relative to which tractions are control inputs.
```

```
%% Finite-dimensional PHS model

method   = 'mFEM';   %  sFEM: Hamilton's principle |   mFEM: Hu-Washizu principle
TAG_mFEM = 'C-C';    % 'C-C': strains continuous   |  'C-D': strains discontinuous

model = femPHS2D(prop,meshFEM,TAG_mFEM);

J  = model.(method).J;
Q  = model.(method).Q;
G  = model.(method).G;
```

Command Window

```
model =
  struct with fields:

       sFEM: [1×1 struct]
       mFEM: [1×1 struct]
    TAG_mFEM: 'C-C'
       coor: [79×2 double]
         NE: 108
         NN: 79
     N_free: 219
    N_Omega: 237
    M_Omega: 395
    idx_sort: [237×1 double]
```

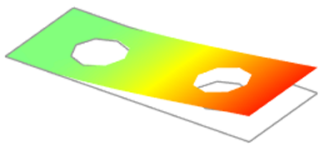
```
%% Eigenvalues and "eigenmodes"

lam = eig(full(J*Q))/2/pi; % Eigenvalues  [Hz]

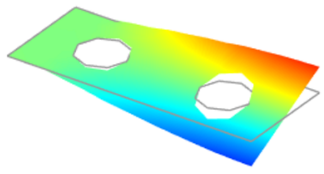
imag_lam = round(imag(lam),2);
wn_fem   = sort(imag_lam(imag_lam>0))';

N_modes = 9;
modesMindlinPlate(model,meshFEM,method,N_modes);
```

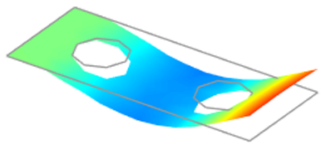
Mode 1: 0.9 [Hz]



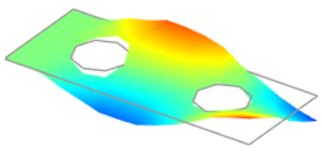
Mode 2: 4.52 [Hz]



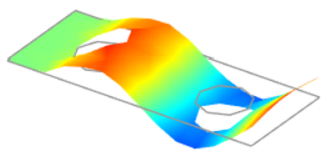
Mode 3: 5.69 [Hz]



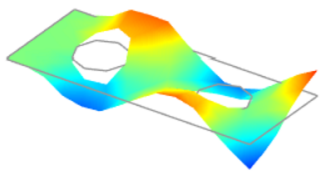
Mode 4: 14.01 [Hz]



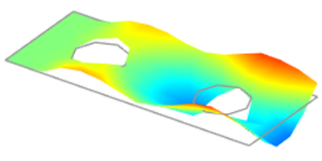
Mode 5: 16.3 [Hz]



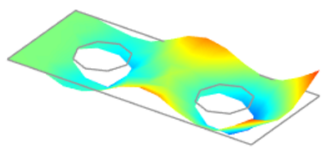
Mode 6: 27.44 [Hz]



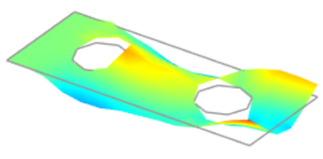
Mode 7: 31.2 [Hz]



Mode 8: 35.05 [Hz]



Mode 9: 42.32 [Hz]



```
%% Time simulation

tspan = [0 1];   dt = 2e-4;
time  = tspan(1):dt:tspan(2);

% Input
AMP  = 10*0.05;   freq = 2*pi*15;    tref = 0.25;
u    = @(t) ones(size(G,2),1).*(AMP*freq*sin(freq*t).*(t<tref));

% Initial conditions
x0 = zeros(size(J,1),1);
x  = zeros(length(x0),length(time));
H  = zeros(length(time),1);

% Implicit midpoint rule
Id = speye(size(J,1));
A = (dt/2)*J*Q;
LHS_dec = decomposition(Id-A);
RHS_mat = Id + A;

for k = 1:length(time)
    H(k) = 0.5*x(:,k)'*Q*x(:,k);
    if k < length(time)
        u_mid = u(time(k) + dt/2);
        rhs = RHS_mat*x(:,k) + dt*(G*u_mid);
        x(:,k+1) = LHS_dec\rhs;
    end
end
```

Thank you for your attention.

Any questions?

cristobal.ponces@usm.cl

Additional slides

Eigenmodes

Consider a discretized PHS model:

$$\underbrace{\begin{bmatrix}\dot{\hat{p}}_\Omega\\ \dot{\hat{r}}_D\\ \dot{\hat{r}}_\Omega\end{bmatrix}}_{\dot{\hat{x}}} = \underbrace{\begin{bmatrix}-\hat{D}_{\Omega\Omega} & 0 & -I_\Omega\\ 0 & 0 & 0\\ I_\Omega & 0 & 0\end{bmatrix}}_{(\hat{J}-\hat{R})} \underbrace{\begin{bmatrix}\hat{M}_{\Omega\Omega}^{-1}\hat{p}_\Omega\\ \hat{K}_{DD}\hat{r}_D+\hat{K}_{D\Omega}\hat{r}_\Omega\\ \hat{K}_{\Omega D}\hat{r}_D+\hat{K}_{\Omega\Omega}\hat{r}_\Omega\end{bmatrix}}_{\nabla\hat{H}(\hat{x})} + \underbrace{\begin{bmatrix}\hat{B}_N & (\hat{D}_{\Omega\Omega}\hat{M}_{\Omega\Omega}^{-1}\hat{M}_{\Omega D}-\hat{D}_{\Omega D})\\ 0 & I_D\\ 0 & -\hat{M}_{\Omega\Omega}^{-1}\hat{M}_{\Omega D}\end{bmatrix}}_{\hat{G}_\partial} \underbrace{\begin{bmatrix}\tau_N\\ v_D\end{bmatrix}}_{u_\partial}$$

$$y_\partial = \hat{G}_\partial^\top \nabla\hat{H}(\hat{x})$$

$$\hat{H}(\hat{x}) = \frac{1}{2}\hat{p}_\Omega^\top \hat{M}_{\Omega\Omega}^{-1}\hat{p}_\Omega + \frac{1}{2}\hat{r}^\top \hat{K}_r \hat{r}$$

Eigenmodes

Consider a discretized PHS model:

$$\underbrace{\begin{bmatrix}\dot{\hat{p}}_\Omega\\ \dot{\hat{r}}_D\\ \dot{\hat{r}}_\Omega\end{bmatrix}}_{\dot{\hat{x}}} = \underbrace{\begin{bmatrix}-\hat{D}_{\Omega\Omega} & 0 & -I_\Omega\\ 0 & 0 & 0\\ I_\Omega & 0 & 0\end{bmatrix}}_{(\hat{J}-\hat{R})} \underbrace{\begin{bmatrix}\hat{M}_{\Omega\Omega}^{-1}\hat{p}_\Omega\\ \hat{K}_{DD}\hat{r}_D+\hat{K}_{D\Omega}\hat{r}_\Omega\\ \hat{K}_{\Omega D}\hat{r}_D+\hat{K}_{\Omega\Omega}\hat{r}_\Omega\end{bmatrix}}_{\nabla\hat{H}(\hat{x})} + \underbrace{\begin{bmatrix}\hat{B}_N & (\hat{D}_{\Omega\Omega}\hat{M}_{\Omega\Omega}^{-1}\hat{M}_{\Omega D}-\hat{D}_{\Omega D})\\ 0 & I_D\\ 0 & -\hat{M}_{\Omega\Omega}^{-1}\hat{M}_{\Omega D}\end{bmatrix}}_{\hat{G}_\partial} \underbrace{\begin{bmatrix}\tau_N\\ v_D\end{bmatrix}}_{u_\partial}$$

$$y_\partial = \hat{G}_\partial^\top \nabla\hat{H}(\hat{x})$$

$$\hat{H}(\hat{x}) = \frac{1}{2}\hat{p}_\Omega^\top \hat{M}_{\Omega\Omega}^{-1}\hat{p}_\Omega + \frac{1}{2}\hat{r}^\top \hat{K}_r \hat{r}$$

The response under free vibration ($u_\partial = 0$) is characterized by:

$$\underbrace{\begin{bmatrix}\dot{\hat{p}}_\Omega\\ \dot{\hat{r}}_D\\ \dot{\hat{r}}_\Omega\end{bmatrix}}_{\dot{\hat{x}}} = \underbrace{\begin{bmatrix}-\hat{D}_{\Omega\Omega} & 0 & -I_\Omega\\ 0 & 0 & 0\\ I_\Omega & 0 & 0\end{bmatrix}}_{(\hat{J}-\hat{R})} \underbrace{\begin{bmatrix}\hat{M}_{\Omega\Omega}^{-1} & 0 & 0\\ 0 & \hat{K}_{DD} & \hat{K}_{D\Omega}\\ 0 & \hat{K}_{\Omega D} & \hat{K}_{\Omega\Omega}\end{bmatrix}}_{\hat{Q}} \underbrace{\begin{bmatrix}\hat{p}_\Omega\\ \hat{r}_D\\ \hat{r}_\Omega\end{bmatrix}}_{\hat{x}} \tag{1}$$

with homogeneous solutions of the form $\hat{x}_H(t) = Ve^{\lambda t}$.

Eigenmodes

Defining $A = (\hat{J} - \hat{R})\hat{Q}$, we substitute $\hat{x}_H(t) = Ve^{\lambda t}$ into (1) to formulate the standard eigenvalue problem:

$$\lambda Ve^{\lambda t} = AVe^{\lambda t} \quad \Longrightarrow \quad (A - \lambda I)V = 0$$

Eigenmodes

Defining $A = (\hat{J} - \hat{R})\hat{Q}$, we substitute $\hat{x}_H(t) = Ve^{\lambda t}$ into (1) to formulate the standard eigenvalue problem:

$$\lambda V e^{\lambda t} = AVe^{\lambda t} \quad \Longrightarrow \quad (A - \lambda I)V = 0$$

- Since V contains the full state amplitude, we partition it to isolate the generalized displacements part V_r:

$$V = \begin{bmatrix} V_p \\ V_r \end{bmatrix}, \quad \text{where} \quad V_r = \begin{bmatrix} V_{r_D} \\ V_{r_\Omega} \end{bmatrix}$$

Eigenmodes

Defining $A = (\hat{J} - \hat{R})\hat{Q}$, we substitute $\hat{x}_H(t) = Ve^{\lambda t}$ into (1) to formulate the standard eigenvalue problem:

$$\lambda Ve^{\lambda t} = AVe^{\lambda t} \quad \Longrightarrow \quad (A - \lambda I)V = 0$$

- Since V contains the full state amplitude, we partition it to isolate the generalized displacements part V_r:

$$V = \begin{bmatrix} V_p \\ V_r \end{bmatrix}, \quad \text{where} \quad V_r = \begin{bmatrix} V_{r_D} \\ V_{r_\Omega} \end{bmatrix}$$

- For underdamped vibratory modes, eigenvalues and their corresponding eigenvectors appear in complex pairs:

$$\lambda_k = -\xi_k \omega_{n,k} \pm j\omega_{n,k}\sqrt{1 - \xi_k^2} \quad \longrightarrow \quad V_{r,k} = \alpha_k \pm j\beta_k$$

Eigenmodes

Defining $A = (\hat{J} - \hat{R})\hat{Q}$, we substitute $\hat{x}_H(t) = Ve^{\lambda t}$ into (1) to formulate the standard eigenvalue problem:

$$\lambda V e^{\lambda t} = A V e^{\lambda t} \quad \Longrightarrow \quad (A - \lambda I) V = 0$$

- Since V contains the full state amplitude, we partition it to isolate the generalized displacements part V_r:

$$V = \begin{bmatrix} V_p \\ V_r \end{bmatrix}, \quad \text{where} \quad V_r = \begin{bmatrix} V_{r_D} \\ V_{r_\Omega} \end{bmatrix}$$

- For underdamped vibratory modes, eigenvalues and their corresponding eigenvectors appear in complex pairs:

$$\lambda_k = -\xi_k \omega_{n,k} \pm j\omega_{n,k}\sqrt{1-\xi_k^2} \quad \longrightarrow \quad V_{r,k} = \alpha_k \pm j\beta_k$$

- To visualize the physical vibration shape of mode k, we extract the real part of the harmonic response over a complete phase sweep $\theta \in [0, 2\pi]$:

$$\hat{r}_{H,k}(\theta) \propto \text{Re}\left(V_{r,k}\, e^{j\theta}\right) = \alpha_k \cos(\theta) - \beta_k \sin(\theta) \tag{2}$$

Implicit Midpoint Rule

The Implicit Midpoint Rule (IMR) applied to a system $\dot{x} = f(x, t)$ is defined as:

$$\frac{x_{k+1} - x_k}{\Delta t} = f(x_m, t_m)$$

where $x_m = \frac{x_{k+1}+x_k}{2}$ and $t_m = t_k + \frac{\Delta t}{2}$ represent the midpoint state and time, respectively.

Implicit Midpoint Rule

The Implicit Midpoint Rule (IMR) applied to a system $\dot{x} = f(x, t)$ is defined as:

$$\frac{x_{k+1} - x_k}{\Delta t} = f(x_m, t_m)$$

where $x_m = \frac{x_{k+1}+x_k}{2}$ and $t_m = t_k + \frac{\Delta t}{2}$ represent the midpoint state and time, respectively.

Considering a conservative linear PHS:

$$\dot{x}(t) = JQx(t) + Gu(t),$$

replacing the midpoints and solving for x_{k+1}, the recursion takes the form:

$$x_{k+1} = \left(I - \frac{\Delta t}{2} JQ\right)^{-1} \left[\left(I + \frac{\Delta t}{2} JQ\right) x_k + \Delta t\, Gu(t_m)\right] \tag{3}$$

Implicit Midpoint Rule

The Implicit Midpoint Rule (IMR) applied to a system $\dot{x} = f(x, t)$ is defined as:

$$\frac{x_{k+1} - x_k}{\Delta t} = f(x_m, t_m)$$

where $x_m = \frac{x_{k+1}+x_k}{2}$ and $t_m = t_k + \frac{\Delta t}{2}$ represent the midpoint state and time, respectively.

Considering a conservative linear PHS:

$$\dot{x}(t) = JQx(t) + Gu(t),$$

replacing the midpoints and solving for x_{k+1}, the recursion takes the form:

$$x_{k+1} = \left(I - \frac{\Delta t}{2} JQ\right)^{-1} \left[\left(I + \frac{\Delta t}{2} JQ\right) x_k + \Delta t\, Gu(t_m)\right] \tag{3}$$

- **Remark:** This method conserves energy for linear PHS.

Proof of energy conservation (1/2)

- The IMR update equation for the unforced system is:

$$\frac{x_{k+1} - x_k}{\Delta t} = JQ\left(\frac{x_{k+1} + x_k}{2}\right)$$

Proof of energy conservation (1/2)

- The IMR update equation for the unforced system is:

$$\frac{x_{k+1} - x_k}{\Delta t} = JQ\left(\frac{x_{k+1} + x_k}{2}\right)$$

- To relate this to the system's energy, we left-multiply both sides of the above equation by $\left(Q\frac{x_{k+1}+x_k}{2}\right)^\top$:

$$\left(Q\frac{x_{k+1} + x_k}{2}\right)^\top \frac{x_{k+1} - x_k}{\Delta t} = \left(Q\frac{x_{k+1} + x_k}{2}\right)^\top JQ\left(\frac{x_{k+1} + x_k}{2}\right)$$

Proof of energy conservation (1/2)

- The IMR update equation for the unforced system is:

$$\frac{x_{k+1} - x_k}{\Delta t} = JQ\left(\frac{x_{k+1} + x_k}{2}\right)$$

- To relate this to the system's energy, we left-multiply both sides of the above equation by $\left(Q\frac{x_{k+1}+x_k}{2}\right)^\top$:

$$\left(Q\frac{x_{k+1} + x_k}{2}\right)^\top \frac{x_{k+1} - x_k}{\Delta t} = \left(Q\frac{x_{k+1} + x_k}{2}\right)^\top JQ\left(\frac{x_{k+1} + x_k}{2}\right)$$

- Defining $v = Q\frac{x_{k+1}+x_k}{2}$, the right-hand side takes the form $v^\top Jv$.

Proof of energy conservation (1/2)

- The IMR update equation for the unforced system is:

$$\frac{x_{k+1} - x_k}{\Delta t} = JQ\left(\frac{x_{k+1} + x_k}{2}\right)$$

- To relate this to the system's energy, we left-multiply both sides of the above equation by $\left(Q\frac{x_{k+1}+x_k}{2}\right)^\top$:

$$\left(Q\frac{x_{k+1} + x_k}{2}\right)^\top \frac{x_{k+1} - x_k}{\Delta t} = \left(Q\frac{x_{k+1} + x_k}{2}\right)^\top JQ\left(\frac{x_{k+1} + x_k}{2}\right)$$

- Defining $v = Q\frac{x_{k+1}+x_k}{2}$, the right-hand side takes the form $v^\top Jv$.

- Since J is skew-symmetric ($J^\top = -J$), any quadratic form constructed with it evaluates to zero ($v^\top Jv = 0$).

Proof of energy conservation (2/2)

- Therefore, the left-hand side must also be zero:

$$\frac{1}{\Delta t}\left(\frac{x_{k+1}+x_k}{2}\right)^\top Q^\top (x_{k+1}-x_k) = 0$$

Proof of energy conservation (2/2)

- Therefore, the left-hand side must also be zero:

$$\frac{1}{\Delta t}\left(\frac{x_{k+1}+x_k}{2}\right)^\top Q^\top (x_{k+1}-x_k) = 0$$

- Since Q is symmetric ($Q^\top = Q$), we can factor out the constants:

$$\frac{1}{2\Delta t}(x_{k+1}^\top Q + x_k^\top Q)(x_{k+1}-x_k) = 0$$

Proof of energy conservation (2/2)

- Therefore, the left-hand side must also be zero:

$$\frac{1}{\Delta t}\left(\frac{x_{k+1}+x_k}{2}\right)^\top Q^\top (x_{k+1}-x_k) = 0$$

- Since Q is symmetric ($Q^\top = Q$), we can factor out the constants:

$$\frac{1}{2\Delta t}(x_{k+1}^\top Q + x_k^\top Q)(x_{k+1}-x_k) = 0$$

- Distributing the terms yields:

$$\frac{1}{\Delta t}\left(\frac{1}{2}x_{k+1}^\top Q x_{k+1} - \frac{1}{2}x_k^\top Q x_k + \underbrace{\frac{1}{2}x_k^\top Q x_{k+1} - \frac{1}{2}x_{k+1}^\top Q x_k}_{0}\right) = 0$$

Proof of energy conservation (2/2)

- Therefore, the left-hand side must also be zero:

$$\frac{1}{\Delta t}\left(\frac{x_{k+1}+x_k}{2}\right)^\top Q^\top (x_{k+1}-x_k) = 0$$

- Since Q is symmetric ($Q^\top = Q$), we can factor out the constants:

$$\frac{1}{2\Delta t}(x_{k+1}^\top Q + x_k^\top Q)(x_{k+1}-x_k) = 0$$

- Distributing the terms yields:

$$\frac{1}{\Delta t}\left(\frac{1}{2}x_{k+1}^\top Q x_{k+1} - \frac{1}{2}x_k^\top Q x_k + \underbrace{\frac{1}{2}x_k^\top Q x_{k+1} - \frac{1}{2}x_{k+1}^\top Q x_k}_{0}\right) = 0$$

- The cross terms cancel out because they are scalars and equal to their own transpose. Then:

$$\frac{1}{\Delta t}\Big(H(x_{k+1}) - H(x_k)\Big) = 0 \quad \Longrightarrow \quad H(x_{k+1}) = H(x_k) \quad \blacksquare$$

The end.